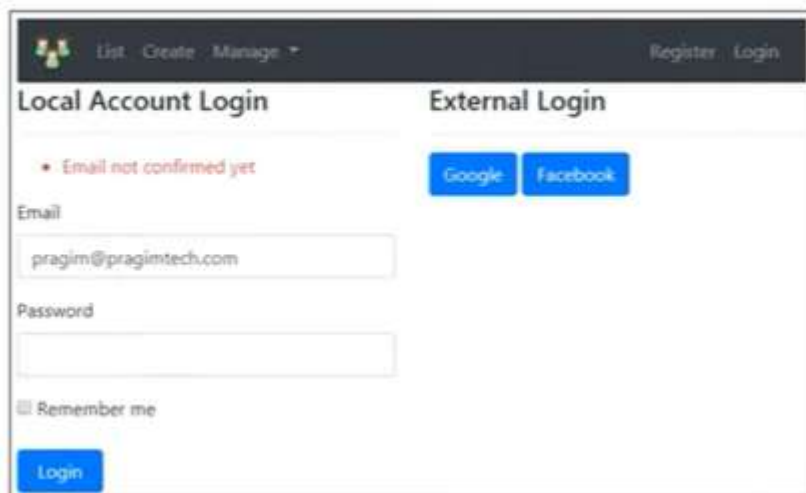


112 - Block login if email is not confirmed

Block Login if Email is Not Confirmed

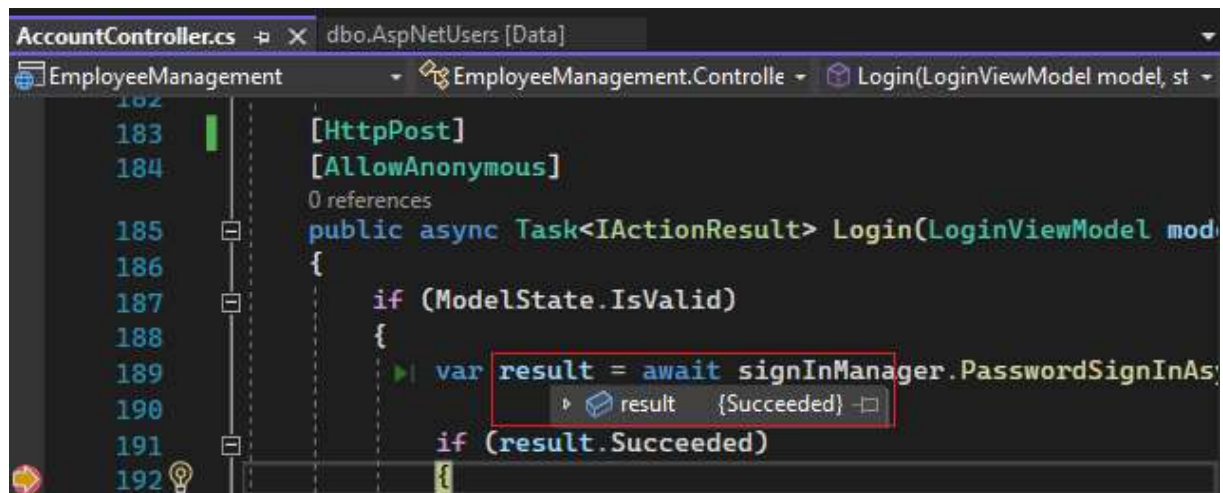


Искаме да се изведе съобщение за грешка при непотвърден имейл при локално или външно вписване.

dbo.AspNetUsers [Data]							
Max Rows: 1000							
	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHas
▶	519-3a32950ab516	nikoleta.panay...	NIKOLETA.PAN...	nikoleta.panay...	NIKOLETA.PAN...	False	NULL
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False	AQAAAAEAA
	84eb9e06-25b7-...	amidalaflame@...	AMIDALAFLAM...	amidalaflame@...	AMIDALAFLAM...	False	NULL
	87cabcf7-6319-...	sara@abv.bg	SARA@ABV.BG	sara@abv.bg	SARA@ABV.BG	False	AQAAAAEAA
	a5b9400f-45cf-...	george@abv.bg	GEORGE@ABV....	george@abv.bg	GEORGE@ABV....	False	AQAAAAEAA
	b14a1adf-9835-...	sofia@abv.bg	SOFIA@ABV.BG	sofia@abv.bg	SOFIA@ABV.BG	False	AQAAAAEAA
	b2056455-c2b4-...	bob@abv.bg	BOB@ABV.BG	bob@abv.bg	BOB@ABV.BG	False	AQAAAAEAA
	cf4120a9-5bca-...	david@abv.bg	DAVID@ABV.BG	david@abv.bg	DAVID@ABV.BG	False	AQAAAAEAA
	d2f456bd-46fc-...	tim@abv.bg	TIM@ABV.BG	tim@abv.bg	TIM@ABV.BG	False	AQAAAAEAA
	f86605f9-0791-...	jason@abv.bg	JASON@ABV.BG	jason@abv.bg	JASON@ABV.BG	False	AQAAAAEAA
⬇	NULL	NULL	NULL	NULL	NULL	NULL	NULL

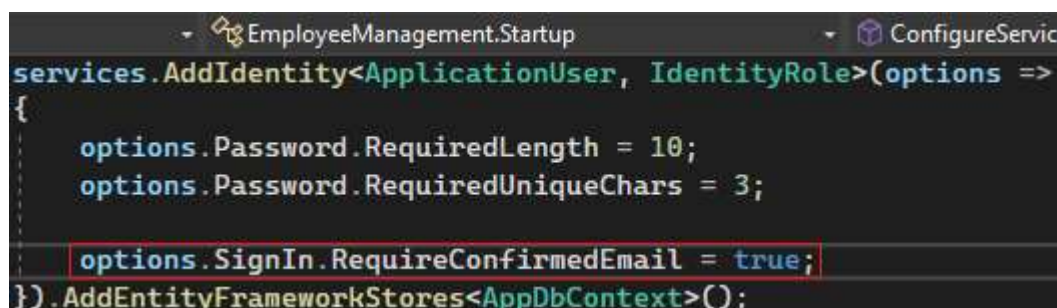
Към момента, в таблицата за локални потребители Users навсякъде EmailConfirmed = false. Това означава, че независимо кой от посочените имейли изберем, ще може да се впишем успешно без потвърждение.

Debug mode -> вписване, изхвърля ни към BreakPoint в Login, AccountController, проверяваме, че преди него променливата result = Succeeded, съответно при Continue ни вписва, като нямаме потвърждение на имейл.



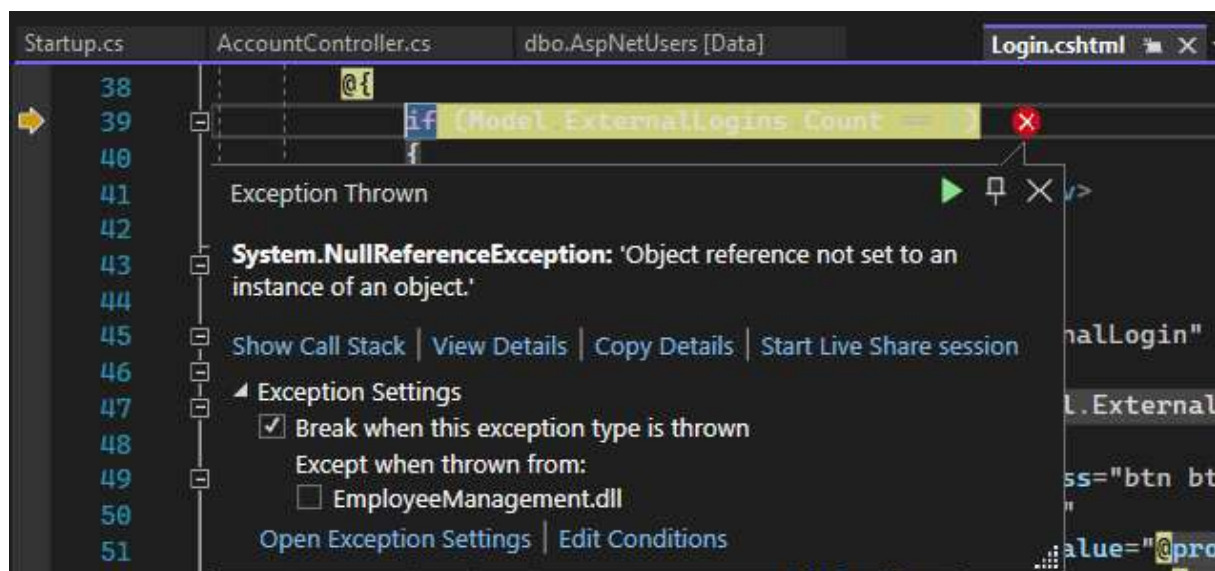
```
AccountController.cs  X  dbo.AspNetUsers [Data]
EmployeeManagement  EmployeeManagement.Controller  Login(LoginViewModel model, st
182
183 [HttpPost]
184 [AllowAnonymous]
0 references
185 public async Task<IActionResult> Login(LoginViewModel model)
186 {
187     if (ModelState.IsValid)
188     {
189         var result = await signInManager.PasswordSignInAsync(model.Email, model.Password, model.IsRemembered, true);
190         if (result.Succeeded)
191         {
192             // ...
193         }
194     }
195 }
```

Единият начин да стане това, е да направим потвърждение на имейл задължително в Startup чрез `options.SignIn.RequireConfirmedEmail = true;`



```
EmployeeManagement.Startup  ConfigureServices
services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.Password.RequiredLength = 10;
    options.Password.RequiredUniqueChars = 3;
    options.SignIn.RequireConfirmedEmail = true;
}).AddEntityFrameworkStores<AppDbContext>();
```

Сега при вписване с Debug ме изхвърли към



```
Startup.cs  AccountController.cs  dbo.AspNetUsers [Data]  Login.cshtml
38
39 if (Model.ExternalLogins.Count == 0)
40 {
41 }
42
43 Exception Thrown
44 System.NullReferenceException: 'Object reference not set to an instance of an object.'
45 Show Call Stack | View Details | Copy Details | Start Live Share session
46
47 Exception Settings
48 [x] Break when this exception type is thrown
49 Except when thrown from:
50 [ ] EmployeeManagement.dll
51 Open Exception Settings | Edit Conditions
```



An unhandled exception occurred while processing the request.

NullReferenceException: Object reference not set to an instance of an object.

AspNetCoreGeneratedDocument.Views_Account_Login.ExecuteAsync() in Login.cshtml, line 39



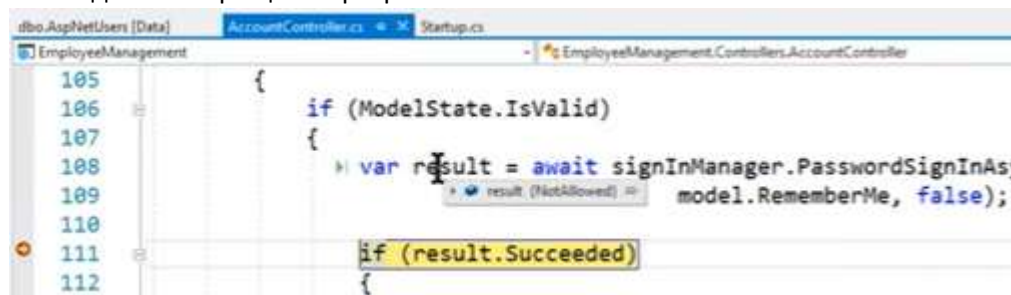
NullReferenceException: Object reference not set to an instance of an object.

AspNetCoreGeneratedDocument.Views_Account_Login.ExecuteAsync() in Login.cshtml

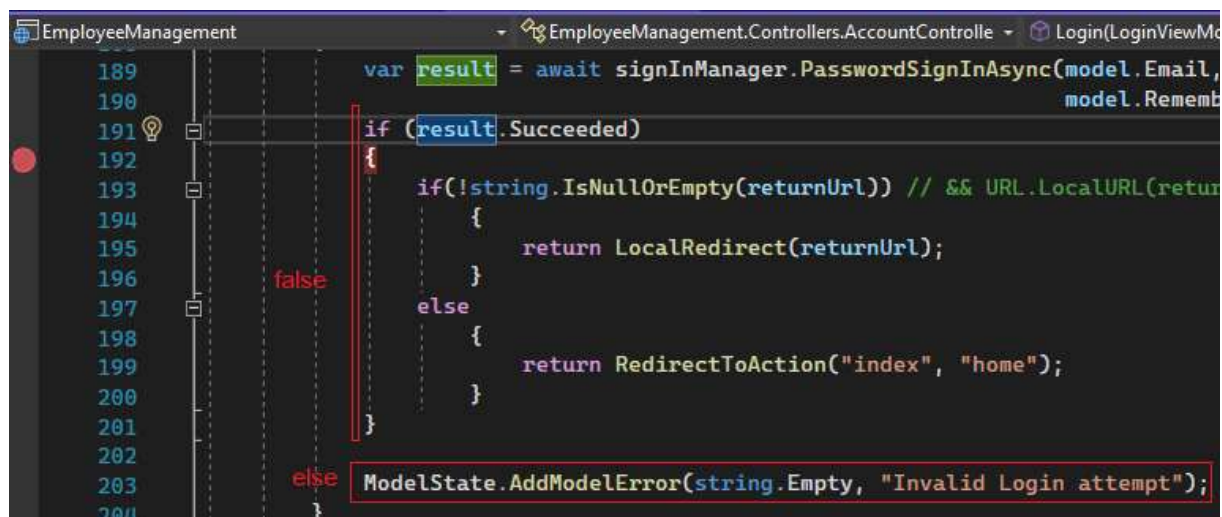
39. if (Model.ExternalLogins.Count == 0)
Microsoft.AspNetCore.Mvc.Razor.RazorView.RenderPageCoreAsync(IRazorPage page, ViewContext context)

if (Model.ExternalLogins.Count == 0)

Във видеото се връща без разрешение в AccountController.



И в двата случая, въпреки че имейлът и паролата са коректни, не разрешава вписване поради непотвърждението на имейла. След Continue извежда същата грешка, въпреки, че очакваното съобщение е при неизпълнението на if-блока „Invalid Login attempt“.



Единият начин да коригираме грешката е директно в Login, за да не бъде Null, извеждаме списък с всички ExternalProviders:

```
model.ExternalLogins =  
(await signInManager.GetExternalAuthenticationSchemesAsync()).ToList();
```

```

EmployeeManagement
EmployeeManagement.Controllers.AccountController
Login(LoginViewModel model, string returnUrl)

183 [HttpPost]
184 [AllowAnonymous]
185 0 references
186 public async Task<IActionResult> Login(LoginViewModel model, string returnUrl)
187 {
188     model.ExternalLogins =
189         (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList();
190
191     if (ModelState.IsValid)
192     {
193         var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
194             model.RememberMe, false);
195         if (result.Succeeded)
196         {
197             // ...
198         }
199     }
200 }

```

Този път при Debug + данни, валидационната грешка я няма и извежда очакваното съобщение: Invalid Login attempt.

Local Account Login

Invalid Login attempt

Email
aurora@abv.bg

Password

☐ Remember me

Login

Но ние не искаме него, а Email not confirmed yet.

```

EmployeeManagement
EmployeeManagement.Controllers.AccountController
Login(LoginViewModel model, string returnUrl)

190 if (ModelState.IsValid)
191 {
192     var user = await userManager.FindByEmailAsync(model.Email);
193     we have found the user and the email is not confirmed
194     if (user != null && !user.EmailConfirmed && against BruteForce
195         (await userManager.CheckPasswordAsync(user, model.Password)))
196     {
197         ModelState.AddModelError(string.Empty, "Email not confirmed yet");
198         return View(model);
199     }
200
201     var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
202         model.RememberMe, false);
203     if (result.Succeeded)
204     {
205         // ...
206     }
207 }

```

```
var user = await userManager.FindByEmailAsync(model.Email);
```

```

if (user != null && !user.EmailConfirmed &&
    (await userManager.CheckPasswordAsync(user, model.Password)))
{
    ModelState.AddModelError(string.Empty, "Email not confirmed yet");
    return View(model);
}

```

Много е важно това съобщение да се изведе едва след проверка за съвпадение между имейл и парола. В противен случай, при брутфорс атака, хакерът ще въвежда рандъм имейли и без проверка за съвпадение на парола, той ще види съобщението, че имейлът не е потвърден и

така ще разбере че той се използва. Той може да изчака няколко дена имейлът да бъде потвърден от истинския собственик и тогава да опита рандъм пароли със същия имейл, за да получи достъп. Ето защо е много важно това съобщение да се извежда едва след сверен имейл и парола.

Сега при тестване, при грешна парола виждаме Invalid login attempt. За хакер това съобщение няма да бъде достатъчно ориентируващо къде е грешката, докато за истински потребител, който знае имейла си, ще насочи към грешна парола. Когато предостави правилна парола, се извежда другата грешка Email not confirmed yet.

Local Account Login Local Account Login

- Invalid Login attempt

Email

Password

☐ Remember me

Login

- Email not confirmed yet

Email

Password

☐ Remember me

Login

Искаме да направим същото из а External Login Providers в ExternalLoginCallback(). Преди да

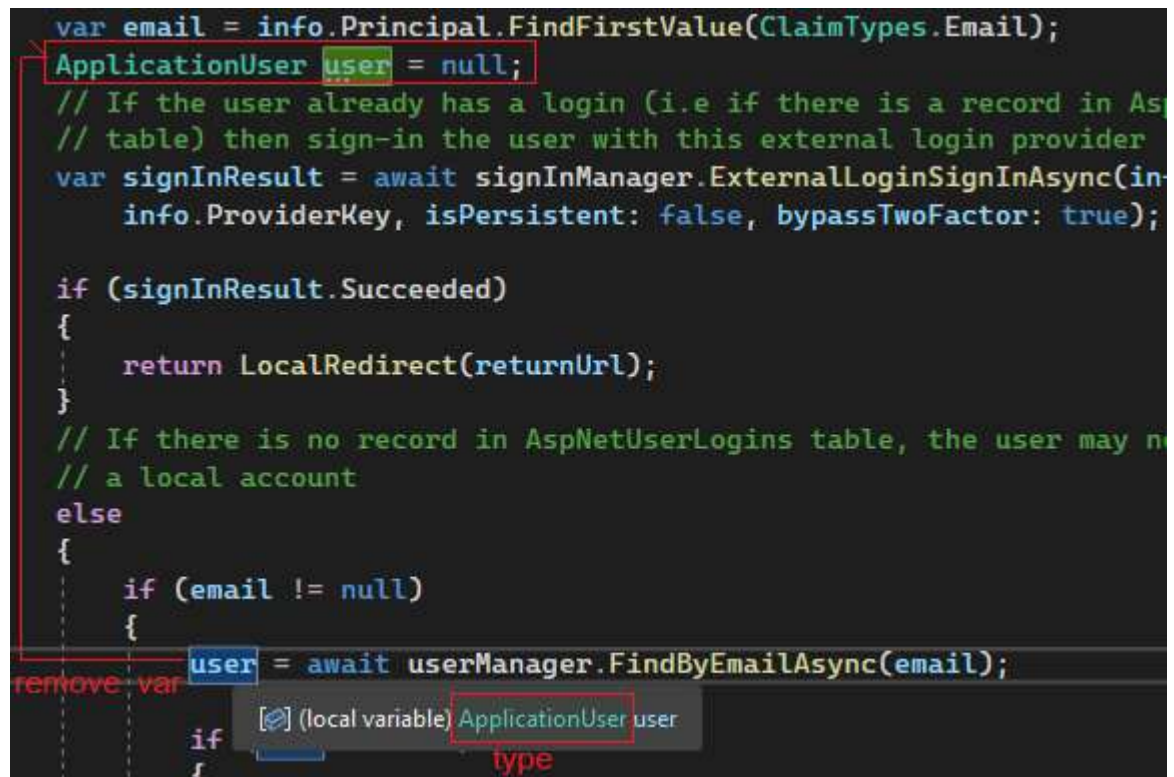
```
EmployeeManagement.Controllers.AccountController ExternalLoginCallback(string returnUrl = null, string...
var info = await signInManager.GetExternalLoginInfoAsync();
if (info == null)
{
    ModelState
        .AddModelError(string.Empty, "Error loading external login information.");
    return View("Login", loginViewModel);
}
// If the user already has a login (i.e if there is a record in AspNetUserLogins
// table) then sign-in the user with this external login provider
var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvider,
    info.ProviderKey, isPersistent: false, bypassTwoFactor: true);

if (signInResult.Succeeded)
{
    return LocalRedirect(returnUrl);
}
// If there is no record in AspNetUserLogins table, the user may not have
// a local account
else
{
    cut
    var email = info.Principal.FindFirstValue(ClaimTypes.Email);
    if (email != null)
```

Преместваме най-

важния ред, от който зависи имейла с Claims между GetLoginInfo - LoginSignIn, т.е. когато разполагаме с информацията и преди да впишем, проверяваме имейла.

```
var user = await userManager.FindByEmailAsync(email);
```



```
var email = info.Principal.FindFirstValue(ClaimTypes.Email);
ApplicationUser user = null;
// If the user already has a login (i.e if there is a record in AspNetUsers table) then sign-in the user with this external login provider
var signInResult = await signInManager.ExternalLoginSignInAsync(info.ProviderKey, isPersistent: false, bypassTwoFactor: true);

if (signInResult.Succeeded)
{
    return LocalRedirect(returnUrl);
}
// If there is no record in AspNetUserLogins table, the user may not have a local account
else
{
    if (email != null)
    {
        user = await userManager.FindByEmailAsync(email);
    }
}
```

remove var [?] (local variable) ApplicationUser user type

Създаваме нова променлива тип ApplicationUser и я зануляваме.

Тогава проверяваме дали съществува такъв имейл и ако да, намираме потребителя, който му съответства.

Ако външният провайдер потвърди потребител с такъв имейл (ние не проверяваме съответствието му с паролата, т.к. те са във външна БД, а получаваме само отговор за потвърждение), връща валидационна грешка за потвърждаване на имейла.

```

ApplicationUser user = null;

if (email != null)
{
    // Find the user
    user = await userManager.FindByEmailAsync(email);

    // If email is not confirmed, display login view with validation error
    if (user != null && !user.EmailConfirmed)
    {
        // in the external DB, so we don't need to check them
        ModelState.AddModelError(string.Empty, "Email not confirmed yet");
        return View("Login", loginViewModel);
    }
}

var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvider,
    info.ProviderKey, isPersistent: false, bypassTwoFactor: true);

if (signInResult.Succeeded)
{
    return LocalRedirect(returnUrl);
}
else
{
    if (email != null)
    {
        // no need to be repeated
        user = await userManager.FindByEmailAsync(email);
        if (user == null)
    }
}

```

При тестване ми позволява вписване през Google, Facebook, а после изписва валидационната грешка, че имейлът не е потвърден.

[AllowAnonymous]

```

public async Task<ActionResult>
    ExternalLoginCallback(string returnUrl = null, string remoteError = null)
{
    returnUrl = returnUrl ?? Url.Content("~/");

    LoginViewModel loginViewModel = new LoginViewModel
    {
        ReturnUrl = returnUrl,
        ExternalLogins =
            (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList()
    };

    if (remoteError != null)
    {
        ModelState.AddModelError(string.Empty,
            $"Error from external provider: {remoteError}");

        return View("Login", loginViewModel);
    }

    var info = await signInManager.GetExternalLoginInfoAsync();
    if (info == null)
    {
        ModelState.AddModelError(string.Empty,
            "Error loading external login information.");
    }
}

```

```

        return View("Login", loginViewModel);
    }

    // Get the email claim from external login provider (Google, Facebook etc)
    var email = info.Principal.FindFirstValue(ClaimTypes.Email);
    ApplicationUser user = null;

    if (email != null)
    {
        // Find the user
        user = await userManager.FindByEmailAsync(email);

        // If email is not confirmed, display login view with validation error
        if (user != null && !user.EmailConfirmed)
        {
            ModelState.AddModelError(string.Empty, "Email not confirmed yet");
            return View("Login", loginViewModel);
        }
    }

    var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvider,
        info.ProviderKey, isPersistent: false, bypassTwoFactor: true);

    if (signInResult.Succeeded)
    {
        return LocalRedirect(returnUrl);
    }
    else
    {
        if (email != null)
        {
            if (user == null)
            {
                user = new ApplicationUser
                {
                    UserName = info.Principal.FindFirstValue(ClaimTypes.Email),
                    Email = info.Principal.FindFirstValue(ClaimTypes.Email)
                };

                await userManager.CreateAsync(user);
            }

            await userManager.AddLoginAsync(user, info);
            await signInManager.SignInAsync(user, isPersistent: false);

            return LocalRedirect(returnUrl);
        }

        ViewBag.ErrorTitle = $"Email claim not received from: {info.LoginProvider}";
        ViewBag.ErrorMessage = "Please contact support on Pragim@PragimTech.com";

        return View("Error");
    }
}

```


Login Action in AccountController

```
[HttpPost]
[AllowAnonymous]
public async Task<IActionResult> Login(LoginViewModel model, string returnUrl)
{
    model.ExternalLogins =
        (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList();

    if (ModelState.IsValid)
    {
        var user = await userManager.FindByEmailAsync(model.Email);

        if (user != null && !user.EmailConfirmed &&
            (await userManager.CheckPasswordAsync(user, model.Password)))
        {
            ModelState.AddModelError(string.Empty, "Email not confirmed yet");
            return View(model);
        }

        // Rest of the code
    }

    return View(model);
}
```

ExternalLoginCallback Action

```
[AllowAnonymous]
public async Task<IActionResult>
    ExternalLoginCallback(string returnUrl = null, string remoteError = null)
{
    LoginViewModel loginViewModel = new LoginViewModel(...);

    var info = await signInManager.GetExternalLoginInfoAsync();
    if (info == null) {...}

    var email = info.Principal.FindFirstValue(ClaimTypes.Email);
    ApplicationUser user = null;

    if (email != null)
    {
        user = await userManager.FindByEmailAsync(email);

        if (user != null && !user.EmailConfirmed)
        {
            ModelState.AddModelError(string.Empty, "Email not confirmed yet");
            return View("Login", loginViewModel);
        }
    }

    // Rest of the code
    return View();
}
```